

GHOSTKEY

PUF-Based Hardware Security Authentication System

Project Overview — Revised & Annotated Edition

Includes clarifications, known limitations, and honest architectural analysis

1. What Is GhostKey?

GhostKey is a hardware security authentication system that uses Physical Unclonable Functions (PUFs), layered tamper detection, and a challenge-response protocol. The core idea is a Hardware Root of Trust — security grounded in hardware rather than software, because any software security is only as strong as the hardware it runs on.

KEY CONCEPT: GhostKey authenticates hardware, not people. It answers the question: "Is this the exact same Arduino that was originally enrolled?" — not "Is this the right person?" This distinction is critical and is explained fully in Section 3.

2. Core Concepts

2.1 Physical Unclonable Function (PUF)

A Physical Unclonable Function exploits the tiny, random physical imperfections that occur during manufacturing of any electronic chip. No two chips are identical at the atomic level — slight variations in resistance, capacitance, and gate delays make every chip physically unique.

How it works — Challenge-Response model:

- Send the chip a challenge (an input)
- The chip produces a response based on its unique physical properties
- The same challenge on a different chip produces a different response
- The hardware itself is the key — there is nothing stored inside the key to steal

SRAM PUF (used in this project):

When an Arduino powers up, its SRAM cells settle into a random pattern of 1s and 0s before firmware initializes them. This pattern is determined by the physical imperfections of that specific chip and is repeatable — the same chip produces the same pattern every power-up. This pattern is captured and used as the device fingerprint.

WHY IT CANNOT BE CLONED: The PUF cannot be cloned because doing so would require replicating atomic-level manufacturing imperfections — physically impossible. Even the original manufacturer cannot produce two identical chips.

Important: Fuzzy Matching Required

The SRAM PUF on an Arduino is not perfectly stable across temperature and voltage variations — readings can drift slightly between power cycles. You must implement a fuzzy matching tolerance (Hamming distance threshold) rather than exact bit comparison, otherwise you will get false rejections of the legitimate device.

Real-world use:

Military hardware verification, anti-counterfeiting in electronics and pharmaceuticals, IoT device identity, supply chain verification.

2.2 Challenge-Response Authentication

Challenge-Response is an authentication protocol where one party (the verifier) sends a random input (challenge) to another party (the prover), and the prover must produce the correct output (response) to prove identity.

In GhostKey:

1. The ESP32 generates a random challenge and sends it to the Arduino
2. The Arduino's PUF processes the challenge using its physical fingerprint
3. The ESP32 compares the response against its enrolled database
4. A match grants access; a mismatch permanently denies it until re-enrollment

IMPORTANT: The ESP32 must generate a truly random challenge every single time and never reuse challenges. If challenges are reused, an attacker who eavesdrops on the UART line can record a valid challenge-response pair and replay it later.

2.3 Hardware Root of Trust

A Hardware Root of Trust means the first thing a system trusts must be hardware, not software — because software can always be modified by an attacker with enough access. Hardware that is physically tamper-resistant provides a foundation that software security can be built on top of.

In GhostKey, the ESP32's eFuse memory and Flash Encryption features provide this root of trust. eFuses are one-time programmable memory cells that store cryptographic keys in a way that is physically unreadable after being written. Once burned, they cannot be changed or extracted.

2.4 Tamper Detection

Tamper detection monitors a device for signs of physical interference and responds automatically — typically by wiping sensitive data or disabling the device. GhostKey uses three layers:

a) Light Tamper Detection (LDR)

A Light Dependent Resistor is placed inside the sealed enclosure. Under normal operation the enclosure is dark. If anyone opens the enclosure, ambient light floods in, the LDR detects the change, and the ESP32 immediately wipes its enrolled PUF database — permanently locking the device until re-enrolled by an authorized party.

b) Vibration Detection (SW-420)

A vibration sensor detects if someone is physically probing, shaking, or attempting to drill into the device. Sustained or sharp vibration triggers a tamper alert.

c) Voltage Monitoring (LM393 Comparator)

A comparator circuit monitors the ESP32's power supply rail for sudden fluctuations. This detects voltage glitching attacks — a real hardware attack technique where an attacker rapidly disrupts the power supply to cause the processor to skip instructions, potentially bypassing authentication checks.

REAL WORLD: Real-world use: ATMs, voting machines, HSMs (Hardware Security Modules), and military communication equipment all use layered tamper detection.

2.5 What Exactly Gets Erased on Tamper?

This is a common source of confusion. There are two distinct things stored on the ESP32:

The enrolled PUF profile (what gets wiped):

A table of challenge-response pairs recorded during enrollment, stored in the ESP32's flash memory. Without this table, the ESP32 has nothing to compare incoming responses against, so even the legitimate Arduino gets rejected. The lock closes permanently.

The firmware itself (NOT wiped):

The ESP32's actual program — authentication logic, tamper detection code, WiFi dashboard — stays intact. The device doesn't become a brick; it simply becomes an unenrolled device that refuses everyone until an administrator re-enrolls it.

ANALOGY: Think of it like a bouncer with a guest list. The firmware is the bouncer — he stays. The enrolled PUF profile is the guest list — that is what gets shredded on tamper. Without the guest list, the bouncer lets nobody in, even legitimate guests.

2.6 Voltage Glitching / Fault Injection Attacks

Voltage glitching is a hardware attack where an attacker introduces a brief, precise disruption to a chip's power supply or clock signal. This causes the processor to behave incorrectly — skipping instructions, producing wrong outputs, or entering undefined states. Attackers use this to bypass security checks in embedded systems.

GhostKey's voltage monitoring circuit detects abnormal power rail behavior and triggers a tamper response before the attack can succeed.

2.7 ESP32 eFuse and Flash Encryption

eFuses:

One-time programmable memory cells that permanently store cryptographic keys. Once written, they cannot be read back through software or changed. They physically lock configuration into the chip.

Flash Encryption:

Encrypts the contents of the ESP32's flash memory so that even if someone physically removes the flash chip and reads it with external hardware, all they see is encrypted data.

Secure Boot:

Ensures the ESP32 only runs firmware that has been cryptographically signed by a trusted key, preventing an attacker from flashing modified firmware.

2.8 Serial UART Communication with Logic Level Shifting

The Arduino Uno operates on 5V logic while the ESP32 operates on 3.3V logic. Connecting them directly without level shifting can permanently damage the ESP32's GPIO pins over time.

A voltage divider (10kΩ and 20kΩ resistors) is used on the Arduino TX to ESP32 RX line to step down 5V signals to 3.3V. The ESP32 TX to Arduino RX line does not need shifting because Arduino can read 3.3V as HIGH.

3. Architectural Clarity (Critical — Read This)

This section documents key architectural decisions and limitations that were clarified during design review. Understanding these is essential for an honest presentation of the project.

3.1 GhostKey Authenticates Hardware, Not People

COMMON MISCONCEPTION

GhostKey authenticates users who possess the Arduino.

CORRECT UNDERSTANDING

GhostKey authenticates the Arduino device itself. Whoever physically holds the Arduino can authenticate — the system has no concept of a person.

The system answers: "Is this the exact same Arduino hardware that was enrolled?" — not "Is this the right person?" This makes it a device authentication system, not a user authentication system.

For a complete access control system, a human authentication layer (PIN, NFC card, biometric) must be added on top. The PUF layer ensures the machine doing the authentication hasn't been tampered with or replaced. The human layer ensures the right person is present.

3.2 What "Nothing is Stored to Steal" Actually Means

COMMON MISCONCEPTION

Nothing is stored anywhere in GhostKey, so there is nothing to steal.

CORRECT UNDERSTANDING

Nothing is stored inside the Arduino (the key). But the ESP32 (the verifier) stores an enrolled challenge-response database that is very much a target.

The correct statement is:

- The Arduino side stores nothing — the PUF response is generated fresh from physics every power-up
- The ESP32 side stores the enrolled challenge-response pairs in flash memory — this is what the tamper wipe erases

This is still a meaningful security advantage over password systems because the stored data is on the verifier side which you control and protect, and the key device (Arduino) if stolen yields nothing without the ESP32's database. But "nothing is stored" is imprecise and should not be used without qualification.

3.3 Why the Button Outside Makes Sense — Only With a Human Factor

A button that simply triggers PUF authentication on a static Arduino inside a fixed box is functionally equivalent to a button that opens a lock directly. The authentication is always going to succeed because the hardware never changes.

The button becomes meaningful only when combined with a second factor that varies per access attempt:

- A PIN the user enters on a keypad — now the button triggers PUF + PIN, both must pass
- **An NFC card tap** — the user presents a card, and the system verifies both the card and the hardware platform
- A time window — authentication is only valid during certain hours

PRACTICAL NOTE: For the project demo, a PIN keypad is the simplest addition that makes the user interaction meaningful. Without it, be honest in your presentation that the button is demonstrating the authentication mechanism rather than serving as a real access control.

3.4 Why Use PUF at All If You're Adding PIN or NFC?

This is a fair challenge. A PIN or NFC system alone is sufficient to authenticate a person. The PUF addresses a different and more subtle threat: substitution of the verification hardware itself.

The substitution attack on a pure NFC system:

5. Attacker physically steals your ESP32 + NFC reader
6. Replaces it with an identical-looking fake device
7. The fake device accepts any card and logs credentials, or simply always returns authenticated
8. The legitimate user has no way to detect this

This is a documented real-world attack against ATMs and access control panels, called a skimming or substitution attack.

THE ACTUAL ROLE OF THE PUF: The PUF doesn't authenticate the user — it authenticates the authenticator. It ensures the machine checking your NFC card is the legitimate authorized machine and not an imposter. An ATM does exactly this before accepting your PIN.

3.5 Honest Limitation: Physical Access Assumption

If an attacker has enough physical access to swap out the ESP32 and NFC reader, they also have enough physical access to open the enclosure and steal the Arduino PUF. The same tamper defenses applied to a PUF system could equally protect a simpler NFC-only system.

GhostKey is most defensible as a learning project demonstrating real hardware security concepts, and as a platform for remote/unattended hardware identity verification (IoT, supply chain, military) where there is no human in the loop. For a local physical access control system specifically, a well-protected NFC system achieves similar security at lower complexity.

Being able to articulate this limitation clearly is more impressive than overselling the project.

4. Key Lifecycle — Enrollment, Authentication, De-enrollment

4.1 Physical Layout

The entire system lives inside the sealed enclosure: Arduino, ESP32, all tamper sensors, and the relay. The only things outside are the solenoid lock output, the power input cable, and wireless communications. The user interacts only with the outside of the box.

CRITICAL: If the Arduino is outside the enclosure, an attacker can unplug it, read its SRAM at startup with their own firmware, and clone the PUF response — completely bypassing everything. The LDR wipe only protects you if opening the box is what exposes the Arduino.

4.2 Phase 1 — Enrollment (Done Once)

9. Assemble the system and power on for the first time
10. Arduino boots up — SRAM settles into its unique fingerprint pattern
11. Press the enroll button or send command via serial / web dashboard
12. ESP32 sends a set of challenges to the Arduino
13. Arduino responds based on its physical fingerprint
14. ESP32 stores those challenge-response pairs as the authorized profile in flash
15. Seal the enclosure — enrollment is now locked, system is armed

4.3 Phase 2 — Normal Authentication (Every Access Attempt)

16. User approaches the sealed box
17. User presses the authentication button on the outside of the enclosure
 1. (If PIN is implemented) *User enters PIN on external keypad*
18. ESP32 generates a fresh random challenge and sends it to the Arduino inside
19. Arduino responds based on its SRAM fingerprint
20. ESP32 compares response against enrolled profile using Hamming distance threshold
21. Match → relay triggers → lock opens → RGB LED green
22. No match → access denied → RGB LED red

The user never touches the Arduino. They interact only with the outside of the box.

4.4 Phase 3 — Tamper Event

23. Someone attempts to open the enclosure or trigger vibration / voltage glitch
24. LDR detects light, or vibration sensor fires, or comparator triggers
25. ESP32 immediately wipes the enrolled PUF database from flash
26. All subsequent authentication attempts fail permanently
27. An authorized administrator must physically re-enroll to restore access

SECURITY PROPERTY: The attacker gains nothing from triggering tamper detection except permanently locking everyone out, which gives them no advantage.

4.5 De-enrollment and Re-enrollment

De-enrolling means deleting the Arduino's challenge-response pairs from the ESP32's flash so the ESP32 no longer recognizes it as authorized. The Arduino itself is unchanged — its SRAM fingerprint is permanent and physical — but without a matching profile on the ESP32 side, it cannot authenticate.

De-enrollment methods (in order of security):

- PIN-protected command via web dashboard — most secure, recommended
- Serial command from a connected computer — controlled, requires physical access
- Physical button hold (5 seconds) — simple but less secure

Key lifecycle summary:

Enrollment → Active use → De-enrollment / Revocation → Re-enrollment with new authorized device

This complete lifecycle is what separates a real security system from a toy. If a key is lost or stolen, revocation must be possible.

4.6 Multi-Device Enrollment (Extension)

The current design enrolls one Arduino. The system can be extended to store multiple enrolled profiles in the ESP32's flash memory. During authentication, the ESP32 tests the incoming response against all enrolled profiles and grants access if any match. This is how a door lock with multiple key cards works. For current project scope, single enrollment is sufficient and cleaner to explain.

5. Demo Logistics

5.1 Demonstrating Rejection With a Second Arduino

To show a failure case (unauthorized Arduino rejected), you need to swap the Arduino without triggering the LDR tamper wipe. Two approaches:

Option A — Demo Mode Switch (recommended):

Add a physical toggle switch on the outside of the enclosure that temporarily suppresses tamper detection. Flip to demo mode, swap Arduino, flip back to live mode, press authenticate — rejected. Being upfront that demo mode exists and would be disabled in production is a mature and honest thing to say.

Option B — Side Hatch:

A small removable panel on the side of the enclosure just large enough to unplug and replug the Arduino — but not large enough to expose the LDR to ambient light. This keeps tamper detection active throughout the demo, which is more impressive.

5.2 Enclosure Notes

The plastic project box (₹100–150) in the components list is the enclosure. An Arduino Uno alone is 7×5cm, so fitting both controllers, sensors, OLED, and wiring requires a 15×10×5cm box or larger. Alternatively, use an Arduino Nano or Pro Mini — same SRAM PUF behavior, fraction of the size.

For a project-based learning course, a well-sealed cardboard box is completely acceptable. The enclosure only needs to be dark inside — not fortress-grade.

6. Components List

#	Component	Qty	Cost (₹)	Role
1	ESP32 Development Board	x1	400–500	Central verifier, tamper monitor, WiFi host, relay controller
2	Arduino Uno	x1	300–400	PUF token — SRAM fingerprint is the hardware key
3	LDR (Light Dependent Resistor)	x1	20–30	Detects enclosure being opened
4	SW-420 Vibration Sensor Module	x1	60–80	Detects physical probing or tampering
5	LM393 Comparator Module	x1	30–50	Monitors power rail for voltage glitching attacks

#	Component	Qty	Cost (₹)	Role
6	5V Single Channel Relay Module	x1	60–80	Controls the lock or output being protected
7	Solenoid Lock or LED Array	x1	150–300	The protected output / visual demo
8	0.96" I2C OLED Display	x1	150–200	Live authentication status on device
9	10kΩ Resistors (pack of 10)	x1	20	LDR voltage divider, logic level shifting
10	20kΩ Resistor	x2	10	Logic level shifting (Arduino 5V → ESP32 3.3V)
11	220Ω Resistors (pack of 10)	x1	15	Current limiting for RGB LED
12	RGB LED (Common Cathode)	x1	20	Green=auth, Red=tamper, Blue=standby
13	100nF Ceramic Capacitors	x1	20	Decoupling / power stabilization
14	Full Size Breadboard	x1	80–100	Prototyping
15	Jumper Wires Pack (M-M and M-F)	x1	100–150	Connections between components
16	Small Plastic Project Box	x1	100–150	Houses the system — essential for tamper detection demo
17	USB Power Supply / 5V Adapter	x1	Owned	Powers the system

Budget Summary

₹1540 – ₹2130 (Budget: ₹3000)

7. Pin Connections

7.1 Arduino Uno → ESP32

Pin	Connection	Note
Arduino TX (Pin 1)	→ 10kΩ+20kΩ divider → ESP32 GPIO16	5V stepped to 3.3V
Arduino RX (Pin 0)	→ ESP32 GPIO17 (TX)	No shifting needed
Arduino GND	→ ESP32 GND	Common ground required
Arduino 5V	→ ESP32 VIN	Powers ESP32

7.2 ESP32 GPIO Assignments

GPIO	Role	Note
GPIO16 (RX)	Arduino TX (via voltage divider)	
GPIO17 (TX)	Arduino RX	

GPIO	Role	Note
GPIO34	LDR output	ADC input
GPIO35	SW-420 vibration sensor DO	Digital interrupt
GPIO36	LM393 comparator output	Voltage monitor
GPIO26	Relay IN	
GPIO21	OLED SDA (I2C)	
GPIO22	OLED SCL (I2C)	
GPIO25	RGB LED Red	via 220Ω
GPIO33	RGB LED Green	via 220Ω
GPIO32	RGB LED Blue	via 220Ω

8. Real-World Applications

GhostKey's PUF concept is most applicable where hardware identity must be verified without human involvement:

- Anti-counterfeiting — verifying genuine hardware components in supply chains
- IoT device identity — proving a sensor on a network is the actual authorized device, not an impersonator
- Supply chain security — detecting component swaps during shipping
- Hardware Root of Trust — foundation for secure boot in enterprise systems
- Anti-tamper systems — ATMs, voting machines, data center hardware

HONEST SCOPE: For local physical access control (door locks, lockers), a well-protected NFC or PIN system achieves similar security at lower complexity. GhostKey's strongest real-world case is remote/unattended device identity verification.

9. Presentation Tips

These are the questions most likely to come up from evaluators, and the honest answers to give:

"Why not just use a fingerprint scanner?"

Fingerprints authenticate people. PUFs authenticate hardware. They solve different problems. A fingerprint scanner cannot verify that a server, IoT sensor, or hardware component hasn't been swapped. For local person-authentication, a fingerprint scanner is simpler and more practical — but GhostKey operates at a different layer.

"Why use a PUF if you're already adding a PIN or NFC?"

The PIN or NFC authenticates the person. The PUF authenticates the machine doing the authentication — ensuring it hasn't been physically replaced with a fake that skims credentials. An ATM does exactly this. Without the PUF layer, a substitution attack is possible.

"Can't an attacker just steal the Arduino?"

Yes — and if they have that level of physical access they can also swap the ESP32. The tamper detection raises the cost of attack but doesn't make it impossible. GhostKey is most defensible as a demonstration platform and for remote scenarios where physical security cannot be assumed.

"What happens if tamper detection fires accidentally?"

The enrolled database is wiped and the device locks permanently until re-enrolled. This is the correct security behavior — a false positive is a nuisance, but a missed tamper event is a breach. The threshold for vibration and voltage sensitivity should be tuned carefully during testing.

END OF DOCUMENT